INSTITUTO TECNOLÓGICO DE ESTUDIOS

SUPERIORES DE OCCIDENTE

Práctica #2: Representación de Texto (Parte 1)

por

César Villarreal Hernández, 707560

Minería de Textos

Dr. Víctor Hugo Zaldivar Carrillo

Miércoles 25 de febrero de 2026

I. Objetivo técnico

El objetivo técnico de esta práctica consiste en realizar la representación de diversos textos de forma algebraica, de manera que se representan como “Vector Space Models”. Existen diversas métricas de las cuales podemos medir la similitud entre diferentes documentos como la generación de matrices de Bag of Words, TF-IDF, así como los cálculos de las distancias coseno y euclidianas entre documentos.

II. Características generales

Para poder representar los articulos de The Guardian de formar algebraíca, primero fue necesario generar un vocabulario a partir de estos documentos. Para esto realizó la lectura de todos los articulos y se fue agregando las palabras a un conjunto (set) en python. Esto se debe a que así evitamos que la palabra se repita y a partir de este posteriormente podamos hacer una lista de todas las palabras.

```python
def process_vocabulary(vocab_lim = 500) -> set:
    """
    Function that obtains all the vocabulary from the stored articles from The Guardian.

    Parameters:
        vocab_lim: maximum amount of words that are allowed in the vocabulary

    Return:
        Vocabulary: set
    """
    vocabulary = set()
    texts_path = workspace.get_texts_path()

    for folder in os.listdir(texts_path):
        tmp_path = os.path.join(texts_path,folder)

        # ignore path if it is not a folder
        if not os.path.isdir(tmp_path):
            continue

        if(os.path.exists(tmp_path)):
            for file in os.listdir(tmp_path):
                if file.endswith(".txt"):
                    file_path = os.path.join(tmp_path, file)
                    curr_art = open(file_path, "r",  encoding="utf-8")

                    for line in curr_art.readlines():
                        words = line.strip().split()
                        for word in words:
                            if(len(word) > 1):
                                word = pre_process_word(word)

                                if(len(vocabulary) >= vocab_lim):
                                    return vocabulary
                                else:
                                    if(word.isalpha() and (len(word) > 1)):
                                        vocabulary.add(word)
                    curr_art.close()

    return vocabulary
```

Para asegurarnos que la palabra que se agregó al vocabulario sea consistente en todos los artículos, fue necesario realizar un preprocesamiento del archivo de manera que en todos sea representado de la misma forma.

```python
def pre_process_word(word:str) -> str:
    """
    Preprocessing of the word before adding it to the vocabulary

    Parameters:
        word: str

    Return:
        new_word: str
    """
    word = word.replace(",", "")
    word = word.replace(".", "")
    word = word.replace(")", "")
    word = word.replace("(", "")
    word = word.replace(";", "")
    word = word.replace(":", "")
    word = word.replace("?", "")
    word = word.replace("\"", "")
    word = word.replace("<", "")
    word = word.replace(">", "")
    word = word.replace("“", "")
    word = word.replace"”", "")
    word = word.replace("’", "")
    word = word.replace("[", "")
    word = word.replace("]", "")
    # regex to remove characters with accents
    word = re.sub(r'[^a-zA-Z0-9áéíóúÁÉÍÓÚñÑ]', '', word)
    # set word to lower case
    new_word = word.lower()

    return new_word
```

Una vez se obtuvo el vocabulario se realizar el procesamiento del Bag Of Words, de manera que se realizó la lectura de todos los artículos y se realizó el conteo de frecuencia de cada una de las palabras. A partir de la información obtenida de todos los artículos, se generó un archivo CSV para guardar la matriz de bag of words.

```python
def process_bag_of_words(vocabulary: set, type="binary") -> None:
    """
    Function to generate a bag of words from all the articles retreived from the Guardian, and
    stores it in a CSV file.

    Parameters:
        vocabulary: it is a set containing all the words from all the articles.
        type: it is the kind of bow that will be processed ("binary" or "count").

    Return:
        None
    """
    texts_path = workspace.get_texts_path()
    vocab_list = sorted(list(vocabulary))
    bow_matrix = []
    doc_names = []

    for folder in os.listdir(texts_path):
        tmp_path = os.path.join(texts_path,folder)

        # ignore path if it is not a folder
        if not os.path.isdir(tmp_path):
            continue

        if(os.path.exists(tmp_path)):
            for file in os.listdir(tmp_path):
                if((True == file.endswith(".txt"))):
                    print("processing %s" % file)
                    file_path = os.path.join(tmp_path, file)
                    bow = {word: 0 for word in vocabulary}

                    curr_art = open(file_path, "r",  encoding="utf-8")

                    for line in curr_art.readlines():
                        words = line.strip().split()
                        for word in words:
                            if(len(word) > 1):
                                word = pre_process_word(word)
                                if word in bow:
                                    if("binary" == type):
                                        bow[word] = 1
                                    elif("count" == type):
                                        bow[word] += 1
                                    else:
                                        # set binary as default
                                        bow[word] = 1

                    curr_art.close()
```

Después se utilizó este archivo CSV para realizar el procesamiento de la matriz de TF IDF. Esta función calcula la frecuencia de uso de las palabra usadas en el documento y realiza el calculo del TF IDF; el cual nos permite saber la importancia relativa de cada palabra de un documento en relación a todo el corpus.

```python
def process_tf_idf(bow_df):
    """
    Function to generates the TF IDF matrix.
    It takes the bag of words data frame, calculates the TF IDF matrix and stores it in a CSV file.

    Parameters:
        bow_df: pandas data frame

    Return:
        None
    """
    # inverse document frequency dict
    idf = {}
    # matrix to store TF IDF Matrix
    tfidf_matrix = []
    # number of documents in corpus
    N = bow_df.shape[0]
    # document frequency
    df = (bow_df > 0).sum(axis=0)
    # IDF calculation
    for word in bow_df.columns:
        idf[word] = math.log(N / (1 + df[word]))

    for _, row in bow_df.iterrows():
        tfidf_row = []
        # calculate the total words used
        total_words = row.sum()

        for word in bow_df.columns:
            if(total_words > 0):
                # calculate term frequency in document
                tf = row[word] / total_words
            else:
                # word is not used
                tf = 0

            # calculate tfidf
            tfidf_row.append(tf * idf[word])

        # save resulting calculation for the current document
        tfidf_matrix.append(tfidf_row)

    bag_of_words_df = pd.DataFrame(tfidf_matrix, index=bow_df.index, columns=bow_df.columns)
    output_path = os.path.join(workspace.get_workspace_path(), "output")
    os.makedirs(output_path, exist_ok=True)
    bag_of_words_df.to_csv(os.path.join(workspace.get_workspace_path(), "output", "tf_idf_matrix.csv"), encoding="utf-8")
```

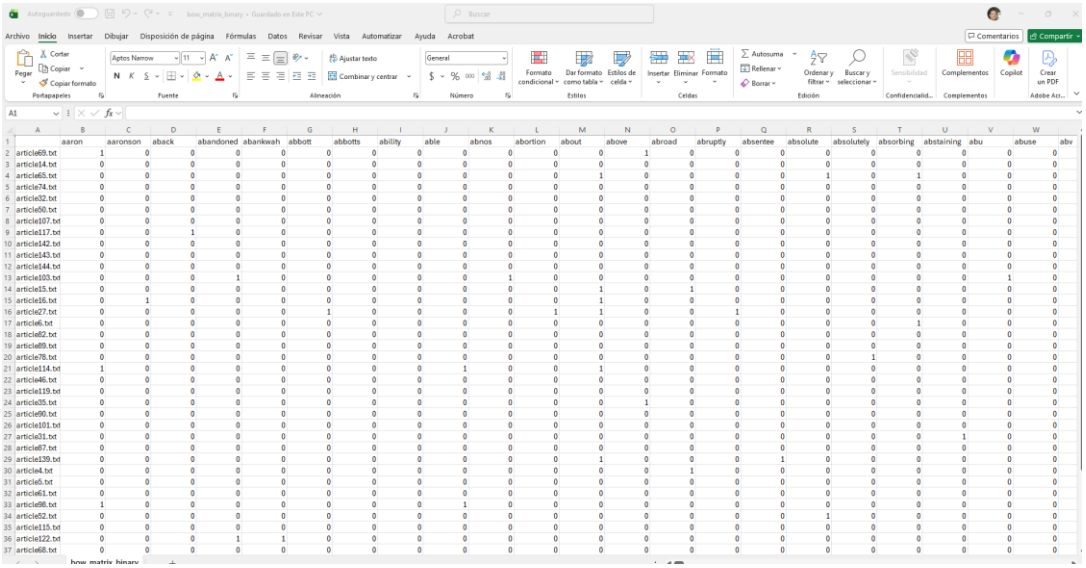
Por último se realiza el procesamiento de las distancias coseno y euclidianas respectivamente.

```python
def process_cosine_distance_matrix(bow_df) -> None:
    """
    Generation of the Cosine Distance Matrix

    Parameters:
        bow_df: Pandas DataFrame

    Return:
        None
    """
    # number of documents
    N = bow_df.shape[0]
    # initialize matrix with zeros
    cos_mat = np.zeros((N, N))
    # retrive vector values
    vectors = bow_df.values

    for i in range(N):
        for j in range(N):
            cos_mat[i, j] = cosine_distance(vectors[i], vectors[j])

    documents_list = bow_df.index.tolist()
    eucl_df = pd.DataFrame(cos_mat, index=documents_list, columns=documents_list)

    output_path = os.path.join(workspace.get_workspace_path(), "output")
    os.makedirs(output_path, exist_ok=True)
    eucl_df.to_csv(os.path.join(workspace.get_workspace_path(), "output", "cosine_dist_matrix.csv"), encoding="utf-8")
```

```python
def process_euclidean_distance_matrix(bow_df) -> None:
    """
    Generation of the Euclidean Distance Matrix

    Parameters:
        bow_df: Pandas DataFrame

    Return:
        None
    """
    # number of documents
    N = bow_df.shape[0]
    # initialize matrix with zeros
    eucl_matrix = np.zeros((N, N))
    # retrive vectorsvector values
    vectors = bow_df.values

    for i in range(N):
        for j in range(N):
            eucl_matrix[i, j] = euclidean_distance(vectors[i], vectors[j])

    documents_list = bow_df.index.tolist()
    eucl_df = pd.DataFrame(eucl_matrix, index=documents_list, columns=documents_list)

    output_path = os.path.join(workspace.get_workspace_path(), "output")
    os.makedirs(output_path, exist_ok=True)
    eucl_df.to_csv(os.path.join(workspace.get_workspace_path(), "output", "euclidean_dist_matrix.csv"), encoding="utf-8")
```

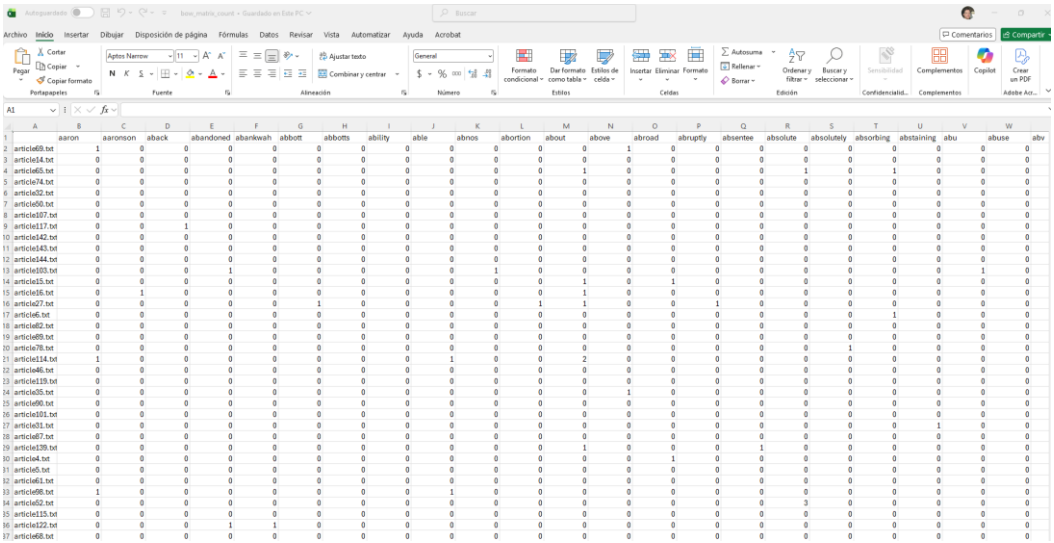
III. Alcances y limitaciones

Posteriormente a calcular la matriz de bag of words binaria, pude darme cuenta de que esta efectivamente nos indica principalmente la presencia y ausencia de diferentes términos. Este resulta útil para saber cuales palabras similares comparten los documentos.

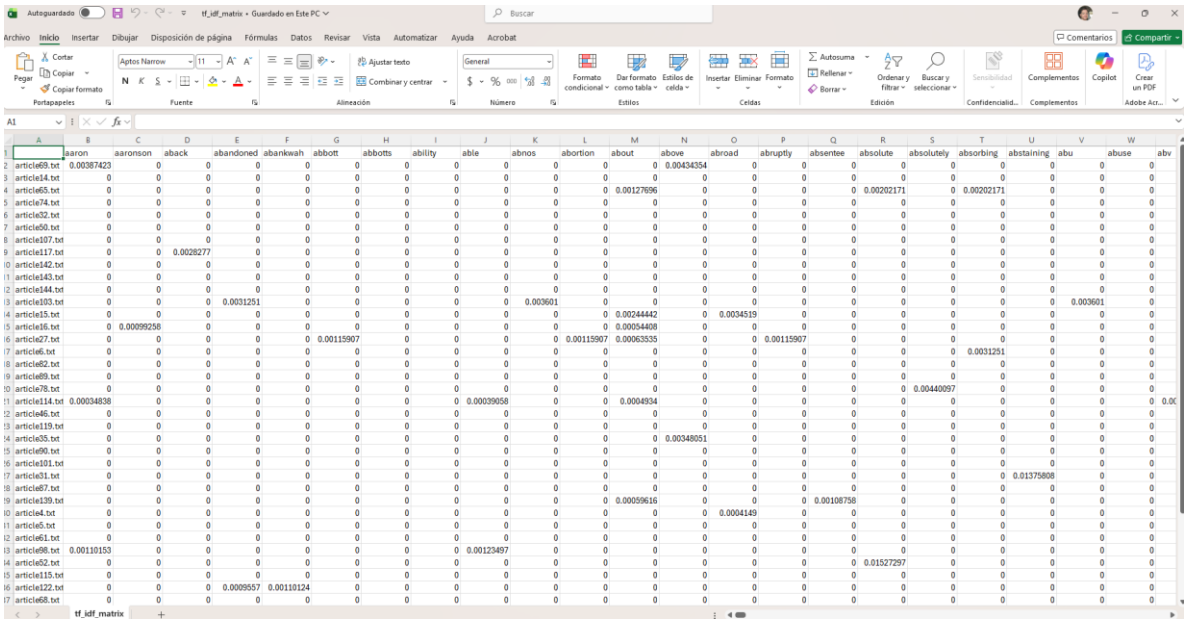


	aaron	aaronson	aback	abandoned	abankwah	abbott	abbotts	ability	able	abnos	abortion	about	above	abroad	abruptly	absentee	absolute	absolutely	absorbing	abstaining	abu	abuse	abv
article69.txt	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
article14.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article65.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	1	0	0	0	0
article74.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article32.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article50.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article107.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article117.txt	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article142.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article143.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article144.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article103.txt	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
article15.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0
article16.txt	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
article27.txt	0	0	0	0	0	1	0	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0
article6.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
article82.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article89.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article78.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
article114.txt	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0
article46.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article119.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article35.txt	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
article90.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article101.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article31.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
article87.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article139.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0
article4.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
article5.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article61.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article98.txt	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article52.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
article115.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article122.txt	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article68.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

De otra manera, el bag of words con conteo resulta más útil dado a que al tener la frecuencia del uso de cada palabra, podemos realizar una mejor identificación de las palabras clave.



	aaron	aaronson	aback	abandoned	abankwah	abbott	abbotts	ability	able	abnos	abortion	about	above	abroad	abruptly	absentee	absolute	absolutely	absorbing	abstaining	abu	abuse	abv
article69.txt	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
article14.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article65.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	1	0	0	0	0
article74.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article32.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article50.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article107.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article117.txt	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article142.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article143.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article144.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article103.txt	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
article15.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0
article16.txt	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
article27.txt	0	0	0	0	0	1	0	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0
article6.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
article82.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article89.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article78.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
article114.txt	1	0	0	0	0	0	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0
article46.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article119.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article35.txt	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
article90.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article101.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article31.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
article87.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article139.txt	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
article4.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
article5.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article61.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article98.txt	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article52.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
article115.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article122.txt	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
article68.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



	aaron	aaronson	aback	abandoned	abankwah	abbott	abbotts	ability	able	abnos	abortion	about	above	abroad	abruptly	absentee	absolute	absolutely	absorbing	abstaining	abu	abuse	abv
article69.txt	0.00387423	0	0	0	0	0	0	0	0	0	0	0	0.00434354	0	0	0	0	0	0	0	0	0	
article14.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article65.txt	0	0	0	0	0	0	0	0	0	0	0	0.00127696	0	0	0	0	0.00202171	0	0.00202171	0	0	0	
article74.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article32.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article50.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article107.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article117.txt	0	0	0.0028277	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article142.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article143.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article144.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article103.txt	0	0	0	0.0031251	0	0	0	0	0	0.003601	0	0	0	0	0	0	0	0	0	0	0.003601	0	
article15.txt	0	0	0	0	0	0	0	0	0	0	0	0.00244442	0	0.0034519	0	0	0	0	0	0	0	0	
article16.txt	0	0.00099258	0	0	0	0	0	0	0	0	0	0.00054408	0	0	0	0	0	0	0	0	0	0	
article27.txt	0	0	0	0	0	0.00115907	0	0	0	0	0.00115907	0.00063535	0	0	0.00115907	0	0	0	0	0	0	0	
article6.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.0031251	0	0	0	
article82.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article89.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article78.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.00440097	0	0	0	0	
article114.txt	0.00034838	0	0	0	0	0	0	0	0.00039058	0	0	0.0004934	0	0	0	0	0	0	0	0	0	0	0.0(
article46.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article119.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article35.txt	0	0	0	0	0	0	0	0	0	0	0	0	0.00348051	0	0	0	0	0	0	0	0	0	
article90.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article101.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article31.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.01375808	0	0	
article87.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article139.txt	0	0	0	0	0	0	0	0	0	0	0	0.00059616	0	0	0	0.00108758	0	0	0	0	0	0	
article4.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0.0004149	0	0	0	0	0	0	0	0	
article5.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article61.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article98.txt	0.00110153	0	0	0	0	0	0	0	0.00123497	0	0	0	0	0	0	0	0	0	0	0	0	0	
article52.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.01527297	0	0	0	0	0	
article115.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article122.txt	0	0	0	0.0009557	0.00110124	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
article68.txt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

IV. Conclusiones

En conclusión, se puede afirmar que se alcanzaron los objetivos principales de esta práctica, de manera que permito comprender y aplicar los fundamentos de la representación algebraica de textos mediante modelos de espacio vectorial. A partir de la construcción del vocabulario, y a generación de matrices de Bag of Words binaria y de conteo, se pudo realizar efectivamente la comparación entre documentos. El calculo de la matriz TF-IDF aportó una visión más precisa sobre la relevancia de los términos dentro del corpus, destacando aquellos que diferencian un texto de otro. Finalmente, la implementación de las métricas de la distancia euclidiana y coseno evidencio que es posible cuantifica la similitud entre documentos.

V. Bibliografía

- Real Academia Española. (s. f.). *CORPES XXI*. https://www.rae.es/banco-de-datos/corpes-xxi